VERSION CONTROL

VERSION CONTROL

Analyses are often non-linear. A bad, but common approach:

- `analysis.ipynb`
- `analysis2.ipynb`
- `analysis3.ipynb`
- `analysis3_2.ipynb`
- `...`

Source version control allows us to:

- maintain a well-documented analysis history,
- experiment with changes risk-free, and,
- create various versions of our code,
- collaborate and share.

GIT FOR VERSION CONTROL

The standard VC tool is `git`

Note: `git` is **separate** from `github`

- `git` is a command-line version control tool,
- github is a website for collaborating on code using git and other tools,
- one can use `git` locally without ever sharing code on github or even creating a github account.

Analogy:

- `git` = MS Word, `github` = a shared google doc

VERSION CONTROL IS FOR *YOU*

Our goals:

1. Exactly reproducible
2. User friendly
3. Transparent
4. Reusable
5. Archived
6. Version controlled

Most of these are for *sharing* your analysis

Version control is mainly *for you*

IT'S WORTH IT

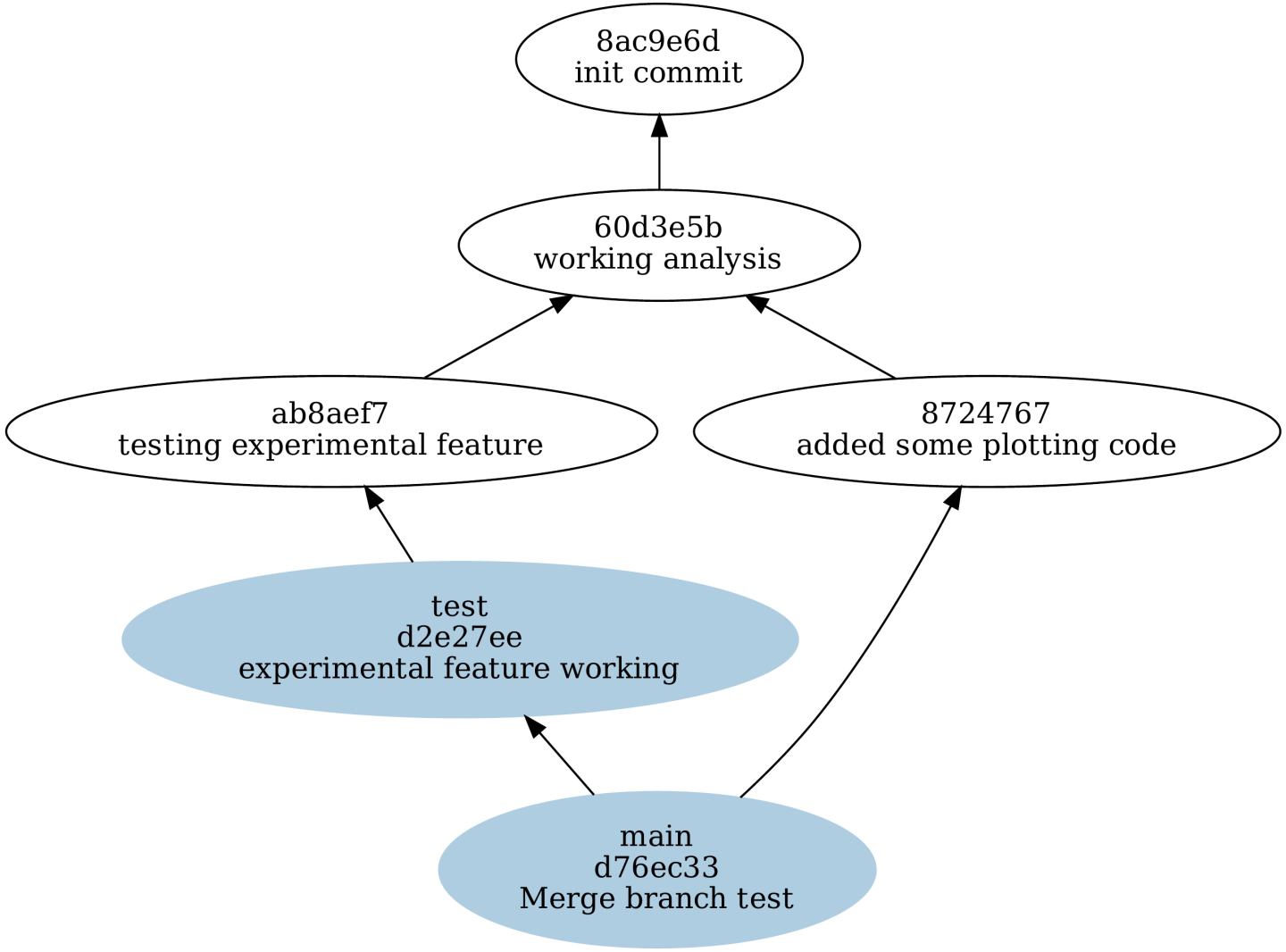
- Git is powerful, and so also fairly complex
- But the basic functionality is pretty simple
- It is well worth learning, and using regularly
- Many, many tutorials online
  - See, e.g., Karl Broman's: https://kbroman.org/github_tutorial/

SNAPSHOTTING

`git` works by periodically taking snapshots of the files (called commits).
Creating snapshots is not automatic, periodically, users need to

1. explicitly state which files to include in the next snapshot, and,
2. manually initiate the snapshot.

SNAPSHOTTING



A MINIMAL TUTORIAL

```
git init -b main
**edit hello.R**
git status
git add hello.R
git status
git commit -m "added init hello.R"
git status
**edit hello.R**
git status
git add hello.R
git status
git commit -m "updated hello.R"
git status
git log --all --oneline --graph
```

```
git checkout HEAD^
less hello.R
```

GRAPHICAL INTERFACES

If you don’t like command-line interfaces, there are lots of graphical interfaces out there, e.g.

- GitHub Desktop
- GitKraken
- Sourcetree

WHAT (NOT) TO SAVE

AN INHERENT TRADE-OFF

You don’t normally save every file in your git repository.

- If you saved everything you did – every edit, every plot, etc. – your git archive would balloon in size
- This happens because git works off files differences (`diff`s)
- If you don’t save enough, you may not be able to piece together what you did

What should you keep?

WHAT TO SAVE

Three useful questions to consider:

- Is the file important?
- Is the file small?
- Do diffs capture its changes in a straight-forward way?

Any “no” answer likely means the file shouldn’t be tracked.

- Unimportant files, especially temporary or easily regenerated ones, shouldn’t clutter the version history.
- Large files can complicate storage and slow usage.
- diffs should provide a straight-forward summary of the changes.

WHAT TO TRACK

`git` was originally designed for source code, any reasonably important, small, plain-text file is likely fine to track.

Examples include:

- source code like `.c`, `.R`, or `.py` files,
- plain `.txt` text files,
- markdown `.md` files,
- configuration files like `.yaml`.

WHAT NOT TO TRACK

- Temporary or intermediate files
  - log files, like .log from latex
  - temporary files, like .tmp
  - history files, like .Rhistory
  - cached editor files, like those in .ipynb_checkpoints

WHAT NOT TO TRACK

- Data, Serialized, or Binary Files
  - large .csv, .tsv, .json, or .xml files,
  - serialized .rds, .pkl, or .hdf5 objects,
  - .dat or .sav files
  - other binary files like .exe

WHAT NOT TO TRACK

- Images and Multi-media
  - .png, .jpeg or .pdf files,
  - music or video files like .mp3, .mp4 or .mkv

WHAT NOT TO TRACK

- Code Libraries or Build Directories
  - Virtual environments (e.g., .venv/ for python)
  - Code libraries (e.g., node_modules/ for node)
  - Automatically created build directories (e.g., _build/ or _book/ for myst and quarto)

WHAT NOT TO TRACK

- Sensitive Data
  - API keys
  - Usernames or passwords
  - Proprietary or controlled data

WHAT NOT TO TRACK

- Can **use jupytext** to mirror to other formats
- e.g. `.R` or `.py` or quarto `.qmd` files are fine to track
- if you **really** want to track `.ipynb` then strip output using `nbstripout` beforehand
  - look up: clean/smudge filters

Note:

- A `.gitignore` file lets you specify what (not) to save
- It is important to set this up at the outset of your project.
- *Once a file has been committed to your repository, it generally can't be removed.*

GITHUB

Github is a collaborative site for sharing version-controlled projects (that are VC with `git`).

- Allows you to share your code/project.
- Allow you to collaborate on code/project.

Note:

- Don’t have to use github, ever. Can just VC local files with git.

DISCUSSION

- What would be your biggest obstacle to using VC?
- What might you do to lessen that obstacle?

EXTRA EXERCISE

Make a **local** git repo for your EDA project. Go back to our minimal example and re-run those commands replacing `hello.R` with your EDA script. Be sure to edit your file and commit an updated version as we did in the example.